Robin Lee

(510) 248-9750 rblee@scu.edu github.com/roblee04 roblee04.github.io linkedin.com/in/robinblee

Education

Santa Clara University — Spring 2026 **GPA 3.97**
B.S / M.S. Computer Science and Engineering, School of Engineering

Experience

Santa Clara University Machine Learning Research Lab – Santa Clara, CA Jun. 2025 — Current
Graduate Researcher

- Projected Toyota's WTS traffic camera footage into 3D using triangulation, multiobject tracking, and ReID to build high-fidelity digital twins in Unreal Engine and NVIDIA Composer.
- Expanded the WTS dataset by **400%** via synthetic data generation using YOLO and NVIDIA IsaacSim.
- Trained generative video-to-video models to synthesize accident data and predict anomalies within future time horizons.
- Trained on multi-GPU clusters (NVIDIA RTX A6000) utilizing CUDA for hardware acceleration.

Santa Clara University IoT Research Lab – Santa Clara, CA
Undergraduate Researcher Jun. 2023 — Sep. 2023

- Built ML models (DB-SCAN, SVM, LSTM) using WiFi spectrum analysis to predict malicious traffic with **70%** recall.
- Simulated active threat flows via Ostinato and NetFlow, and hardened device security with dynamic firewalls and network segmentation.

Santa Clara University Engineering Department – Santa Clara, CA
Graduate Teaching Assistant Aug. 2024 — Current

- Directed 3 weekly labs for **500+ students** covering advanced data structures, memory management, and C programming.
- Instructed **60+ students** in Embedded Systems utilizing C and Thumb ARM Assembly, emphasizing memory alignment, bit-level manipulation, and SIMD optimizations.
- Instructed **100+ students** on Agile methodologies, CI/CD, Git, Docker, and cloud-native architectures (Microservices).

Projects

NUMA-Aware LLM Inference Engine (llama.cpp fork) Jan. 2026 – Mar. 2026
Technologies Used: C, C++, libnuma, Linux kernel APIs (mbind / sched_setaffinity / mlock), GGML, CMake, llama.cpp, AWS EC2

- Engineered a hardware-aware Llama 3.3-70B inference stack on a **192-core** AMD EPYC server, building a custom dual-buffer allocator with mbind to replicate weights across NUMA nodes and eliminate cross-socket memory traffic.
- Designed a strict core-pinning scheduler via sched_setaffinity and a GGML pointer-swap hook to guarantee fully cache-local GEMM execution without modifying the inference API.
- Delivered **3.7×** generation throughput (28.2 → 104.5 tok/s,) and **1.7×** prompt-processing throughput (160 → 268.7 tok/s) on Llama-3-8B benchmarks, sustaining 1.5× gains on 2,048-token contexts.

Bare-Metal Preemptive Task Scheduler Mar. 2025 - May 2025
Technologies Used: C, Inline Assembly, QEMU, Unix/Linux Architecture

- Engineered a custom, priority-based preemptive CPU scheduler for a minimal, UNIX-like kernel environment, successfully managing up to **256 concurrent threads** with a **sub-5 microsecond** context switch overhead.
- Managed hardware timer interrupts and deterministic CPU state transitions using C and inline Assembly.
- Eliminated race conditions during 24+ hour stress tests by integrating custom mutexes, spinlocks, and semaphores.

32-bit Pipelined CPU & Vector Accelerator Mar. 2024 - June 2024
Technologies Used: C, Inline Assembly, Verilog HDL, RTL Simulation (Quartus)

- Designed a 32-bit, 5-stage pipelined CPU in Verilog, supporting 13 instructions and custom conditional branching.
- Engineered hazard detection and data forwarding units to resolve data/control hazards, minimizing pipeline stalls and optimizing CPI
- Accelerated array processing by implementing a custom Vector Addition (ADDV) instruction, mimicking modern AI/SIMD hardware.
- Analyzed RTL waveforms and custom assembly testbenches to verify timing, achieving an **840ns** execution delay.

Applied Auto-tuning on LoRA Hyperparameters Sept. 2023 – Jun. 2024
Technologies Used: llama.cpp, GPTune, QLoRA, Llama3, Google Cloud

- Optimized fine-tuning for Llama3 8B, Qwen2 8B, and Mistral 7B using QLoRA quantization techniques.
- Built a Bayesian auto-tuning pipeline with GPTune to optimize rank, alpha, and learning rate, improving resource efficiency.
- Achieved a **30% reduction in training loss** over a random search baseline without increasing compute time and perplexity.

Skills

Languages: C, C++, ARM, RISC-V, Python, Java, Typescript, SQL, HTML/CSS, Verilog, Bash/Zsh,
Frameworks: CUDA, Flask, Django, FastAPI, SQLAlchemy, Node.js, React.js, Pytorch, Tensorflow
Software Tools: Git, Linux (Arch), Jira, AWS, Docker, Kubernetes, QEMU, Jupyter
Hardware & Debugging: RTL Verification (Quartus), FPGA, Valgrind, GDB, Linux Perf